BALD TEACHER

Cauã Aono · Arthur Carvalho · Nicollas Mota · Lucas Pereira

O CONCEITO E IDEIA INICIAL

"Navegue pelos mapas. Desvie dos alunos. Restaure sua autoestima."

GÊNERO

2D Top-Down Arcade

PLATAFORMA

Navegador Web (HTML5 Canvas)

LOOP PRINCIPAL

Explorar · Desviar Coletar & Evoluir

O Professor Adriano navega pelas salas de aula da universidade coletando cabelos e perucas espalhados pelo chão para restaurar sua autoestima — enquanto desvia dos alunos que querem impedi-lo. Cada fase escala em dificuldade com novos tipos de inimigos e ambientes.

FUNCIONALIDADES

Colisão Avançada

Mapeamento de limites de tela e obstáculos estáticos (mesas, cadeiras) para navegação autêntica.

HUD Estilo Zelda

Sistema de 3 corações e silhuetas de itens coletados que se preenchem conforme o jogador avança.

Gestão de Cenas

Transições fluidas entre Menu Inicial, Fases independentes e Tela de Game Over.

Áudio Integrado

Trilhas sonoras por fase com transições de áudio suaves entre os diferentes níveis.

TECNOLOGIAS UTILIZADAS

JavaScript ES6+	LINGUAGEM BASE ES Modules, classes, sintaxe moderna
Phaser 3	ENGINE GRÁFICA Framework HTML5 robusto para jogos 2D no navegador
Arquitetura POO	DESIGN DE CÓDIGO Entity → Player / Enemies com herança e polimorfismo
HTML5 + CSS3	ESTRUTURA WEB Contêiner de renderização via Canvas
Pixel Art	ESTILO VISUAL Sprites e tilesets criados sob medida

ENGENHARIA: OS 4 PILARES DA POO

1

Abstração

Entidades reais viram classes (Player, Student, Rat) com apenas os atributos essenciais: posição, vida e tamanho de colisão.

2

Encapsulamento

A lógica de dano vive dentro de cada classe. A cena apenas chama um método — as regras internas ficam protegidas.

3

Herança

Entity.js fornece a física. Player.js e Enemies.js a estendem. Todas as fases herdam de Phaser.Scene — reaproveitamento massivo.

4

Polimorfismo

update() é chamado igualmente para todo inimigo, mas a Barata (Fase 2) e os Alunos (Fase 1) reagem de formas completamente diferentes.

Polimorfismo

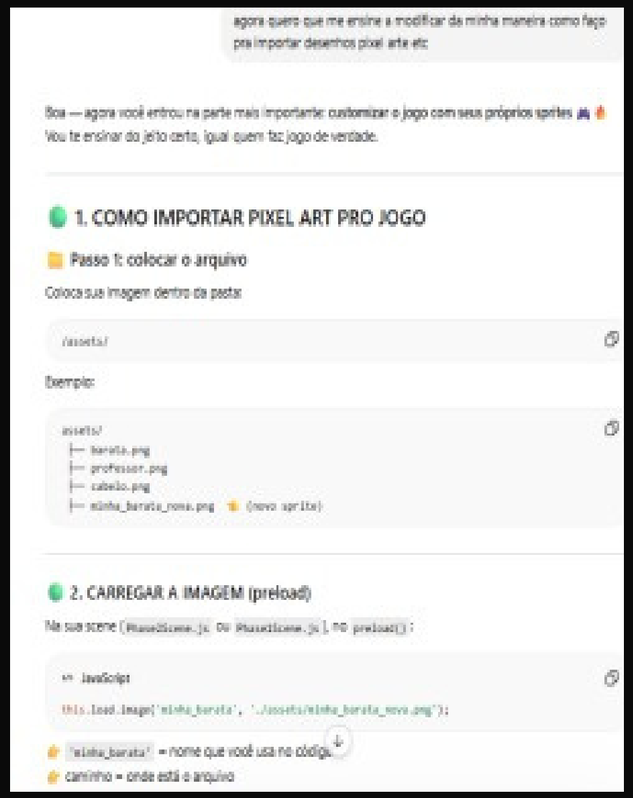
```
JS Phase3Scene.js ●   JS Phase2Scene.js ×
src > scenes > JS Phase2Scene.js > ...
5   export class Phase2Scene extends Phaser.Scene {
22      create() {
114         this.physics.add.collider(this.player, this.rats, this.hitRat, null, this);
115         this.physics.add.collider(this.player, this.obstacles);
116         this.physics.add.collider(this.rats, this.obstacles);
117         this.physics.add.collider(this.rats, this.rats);
118     }
119
120     update() {
121         if (this.player) {
122             this.player.update();
123         }
124
125         this.rats.getChildren().forEach(rat => {
126             rat.update();
127         });
128     }
```

Herança

```
import { Player } from '../entities/Player.js';
import { Rat } from '../entities/Enemies.js';
import { Hair } from '../entities/Hair.js';

export class Phase2Scene extends Phaser.Scene {
    constructor() {
        super('Phase2Scene');
    }
```

CHAT GPT X GEMINI



agora quero que me ensine a modificar da minha maneira como faço pra importar desenhos pixel arte etc

Boa — agora você entrou na parte mais importante: customizar o jogo com seus próprios sprites 🎮🔥
Vou te ensinar do jeito certo, igual quem faz jogo de verdade.

🟢 1. COMO IMPORTAR PIXEL ART PRO JOGO

📁 Passo 1: colocar o arquivo

Coloca sua imagem dentro da pasta:

```
/assets/
```

Exemplo:

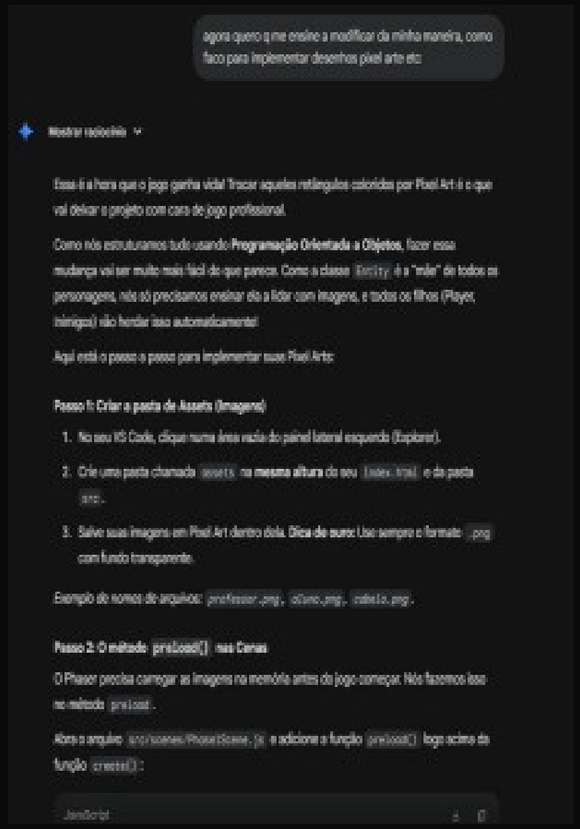
```
assets/
├── barata.png
├── professor.png
├── cabelo.png
├── minha_barata_nova.png   ⭐ (novo sprite)
```

🟢 2. CARREGAR A IMAGEM (preload)

Na sua scene (`PhaseScene.js` ou `MaseScene.js`), no `preload()`:

```
JavaScript
this.load.image('minha_barata', './assets/minha_barata_nova.png');
```

👉 `'minha_barata'` = nome que você usa no código.
👉 caminho = onde está o arquivo



agora quero q me ensine a modificar da minha maneira, como faco para implementar desenhos pixel arte etc

Mostrar raciocínio

Essa é a hora que o jogo ganha vida! Trocar aquelas retângulos coloridos por Pixel Art é o que vai deixar o projeto com cara de jogo profissional.

Como nós estruturamos tudo usando **Programação Orientada a Objetos**, fazer essa mudança vai ser muito mais fácil do que parece. Como a classe `Entity` é a "mãe" de todos os personagens, nós só precisamos ensinar ela a lidar com imagens, e todos os filhos (Player, inimigos) vão herdar isso automaticamente!

Aqui está o passo a passo para implementar suas Pixel Arts:

Passo 1: Criar a pasta de Assets (Imagens)

1. No seu VS Code, clique numa área vazia do painel lateral esquerdo (Explorer).
2. Crie uma pasta chamada `assets` **na mesma altura** do seu `index.html` e da pasta `src`.
3. Salve suas imagens em Pixel Art dentro dela. **Dica de ouro:** Use sempre o formato `.png` com fundo transparente.

Exemplo de nomes de arquivos: `professor.png`, `aluno.png`, `cabelo.png`.

Passo 2: O método `preload()` nas Cenas

O Phaser precisa carregar as imagens na memória antes do jogo começar. Nós fazemos isso no método `preload`.

Abra o arquivo `src/scenes/PhaseScene.js` e adicione a função `preload()` logo acima da função `create()`:

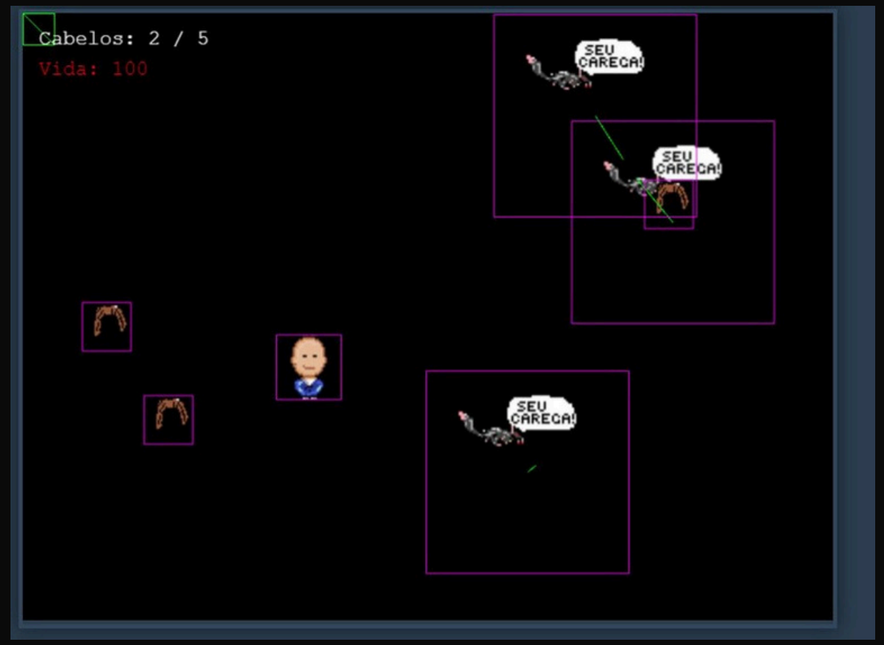
JavaScript

Diferença

ChatGPT: Tende a dar respostas muito diretas e estritamente focadas no bloco de código. É excelente para gerar algoritmos matemáticos complexos de forma fria.

Gemini: Tem um perfil mais analítico e voltado à resolução de problemas complexos com contexto. Ele tende a explicar o porquê o código quebrou (ex: conflito de lógica de Game Design ou arquitetura do Phaser) em vez de só cuspir a resposta.

ANTES E DEPOIS



Documentação de todos os detalhes tecnicos e comparativo de resposta entre duas IAs em arquivo pdf.

OBRIGADO!

Cauã Aono Laredondo

Arthur Carvalho Chagas

Nicollas Mota

Lucas Pereira